

The Send Window

הגענו לשלב שבו אנחנו הופכים מהגדרות תיאורטיות לניהול המכני המדויק בתוך הקרנל. ה-Sending Window (או במובנו המדויק יותר ב-TCP, Send Window) הוא המנגנון שקובע כמה נתונים השולח יכול להזרים לרשת בכל רגע נתון.

הגדרה: מהו ה-Sending Window?

ה-Sending Window הוא "טווח הרצפים" (Sequence Number Range) שהשולח מורשה להעביר ברשת מבלי לקבל אישור (ACK).

מתמטית, הוא מוגדר כ:

$$Window = LastByteAcked + min(rwnd, cwnd) - LastByteSent$$

במילים פשוטות: זהו המרחק בין המידע האחרון שקיבלנו עליו אישור (ACK), לבין המקום שבו אנחנו "עוצרים" כי הגיעו למגבלה של הזיכרון אצל הנמען (rwnd) או המגבלה של העומס ברשת (cwnd).

איך הוא ממומש בתוך ה-Sending Buffer?

- ה-Sending Buffer אינו סתם "מחסן" של בטים. זהו מבנה נתונים מורכב (Linked List של סגמנטים) המחולק ל-4 אזורים לוגיים:
1. **אזור שנשלח ואושר (Acknowledged):** נתונים שנחקקים מהזיכרון או מסומנים כפנויים.
 2. **אזור שנשלח אך טרם אושר (In-Flight):** נתונים שנמצאים "על החוט". הם חייבים להישאר בזיכרון (Sending Buffer) למקרה שנצטרך לבצע שידור חוזר (Retransmission).
 3. **אזור שניתן לשליחה (Usable):** הנתונים שהאפליקציה כתבה, אבל טרם שלחנו לרשת. הגבול של האזור הזה הוא ה-Sending Window.
 4. **אזור חסום (Blocked/Locked):** נתונים שהאפליקציה רצתה לשלוח, אבל ה-Sending Window התמלא. האפליקציה תישאר חסומה ב-write() עד שיתפנה מקום.

איפה המידע שמור ומנוהל?

- ב-Linux 6.14, המידע מנוהל בתוך ה-struct sock (הייצוג של הסוקט בקרנל):
- **מבני הנתונים:** ה-Kernel משתמש ב-sk_write_queue, שהיא תור (Queue) של sk_buff, מבנה נתונים שנקרא SKB. כל SKB מחזיק מצביע לנתונים בזיכרון, גודל, וזמני שליחה.
 - **הניהול:** ישנם משתנים אטומיים בתוך ה-TCP Control Block (tcp_sock) שעוקבים אחרי:
 - snd_una : Sequence Number של הבית הראשון שלא אושר.
 - snd_nxt : Sequence Number של הבית הבא שישלח.
 - snd_wnd : גודל החלון כפי שדווח מהצד השני.
 - **המקום:** הכל ב-Kernel Memory, ספציפית ב-slabs (הקצאת זיכרון יעילה של הליבה).

הטיימרים (Timers) – "שומרי הסף"

- בלינוקס 6.14, הניהול של ה-Sending Window מונע ע"י מספר טיימרים קריטיים:
1. **RTO (Retransmission Timeout):** הטיימר הכי חשוב. אם לא קיבלנו ACK לנתונים שנמצאים באזור ה-In-Flight בזמן המוגדר, אנחנו מניחים אובדן.
 2. **Persistent Timer:** אם ה-rwnd היה אפס, אנחנו צריכים מדי פעם "לבדוק" (Zero Window Probe) אם הנמען התפנה. הטיימר הזה אחראי לשלוח את ה-Probe.
 3. **Keepalive Timer:** מוודא שהחיבור עוד "חיי" גם כשאין תעבורה.
 4. **Tail Loss Probe (TLP):** מנגנון מודרני ב-Linux. אם סגמנט אחרון בתור לא קיבל ACK, ה-TLP יפעיל שידור חוזר מוקדם מאוד (לפני ה-RTO הארוך) כדי למנוע המתנה מיותרת.

הקשר ל-Linux 6.14:

בקרנל המודרני, הטיימרים הללו אינם טיימרים "טיפשים". הם משתמשים ב-hrtimer (High Resolution Timers). זה מאפשר ללינוקס לדייק ברמת המיקרו-שנייה. אם ה-RTO המחושב הוא 10 ms, הקרנל יפעיל את פסיקת ה-RTO בדיוק בזמן, מה שמאפשר ל-TCP להתאושש מהר מאוד מאובדן נתונים.

סיכום: ה-Sending Window הוא הדינמיקה שבין ה-sk_write_queue (הנתונים שממתינים לשליחה) לבין ה-tcp_sock (משתני הבקרה המחשבים את גבולות השליחה).

סיכום הקשר שבין: חלון השליחה והניהול שלו, - מנגנון בקרת הזרימה, ו- מנגנון בקרת העומסים

זהו הסיכום האסטרטגי שמאחד את כל החלקים שניתחנו.
סיכום זה מסביר את "הריקוד המשולש" של TCP: הלקוח (הנמען), הרשת (הצינור), והשרת (השולח).

הריקוד המשולש: סינכרון בין זיכרון, עומס וזרימה
ה-TCP Sender ב-Linux 6.14 אינו סתם "מעביר נתונים". הוא מכונת החלטות מבוזרת המנסה לפתור אופטימיזציה תחת אילוצים משתנים. הסינכרון בין חלון השליחה לבין מנגנוני הבקרה הוא שמאפשר לאינטרנט המודרני לתפקד.

1. חלון השליחה כ"מסגרת העבודה" (The Framework)
ה-Sending Window אינו ערך תיאורטי; הוא הגבול הפיזי של ה-sk_write_queue. הוא מוגדר בכל רגע על ידי:
$$\text{Window} = \min(\text{cwnd}, \text{rwnd})$$

ה-cwnd (בקרת עומסים) הוא החיישן שלנו לתקינות הרשת (האם יש מקום בנתבים?), בעוד שה-rwnd (בקרת זרימה) הוא החיישן שלנו לקיבולת הנמען (האם יש מקום בזיכרון הליבה של הנמען?).
הניהול של ה-Sending Window מתבצע על ידי עדכון מתמיד של מצבי ה-SKB (Socket Buffers) בתוך הקרנל.
כאשר ה-Windows מתכווץ, הנתונים הופכים מ"מוכנים לשליחה" ל"חסומים", והאפליקציה (ה-User Space) נעצרת ב-send() עד שהקרנל יפתח את החלון מחדש.

2. בקרת הזרימה (Flow Control) כ"מנגנון הגנה מקומי"
בקרת הזרימה, המנוהלת ע"י ה-rwnd, פועלת כבלם חירום. היא מתבססת על ה-sk_rcvbuf של הנמען. ב-Linux 6.14, היא פועלת בסינכרון הדוק עם ה-Wait Queue של התהליכון המקבל: ברגע שה-rwnd מגיע לסף מסוכן, המערכת "דוחפת" את האפליקציה לפנות נתונים. כאן אין ניסוי וטעייה – זהו חוזה דטרמיניסטי: הנמען מצהיר בדיוק כמה בתים הוא מוכן לקבל, והשולח מחויב לציית.

3. בקרת העומסים (Congestion Control) כ"מנגנון ניהול סיכונים גלובלי"
בעוד שבקרת הזרימה דטרמיניסטית, בקרת העומסים (cwnd) היא סטטיסטית ומנבאת. ב-Linux 6.14, באמצעות BBR או Cubic, המערכת מנהלת ניסוי מתמיד. היא מגדילה את ה-cwnd עד שהיא מזהה "התנגדות" (בצורה של איבוד חבילות או עלייה ב-RTT).

הסינכרון כאן הוא קריטי :

- אם ה-cwnd קטן משמעותית מה-rwnd, הבעיה היא ברשת.
- אם ה-rwnd קטן משמעותית מה-cwnd, הבעיה היא בשרת המקבל (הוא איטי).

ה-Kernel משתמש ב-TCP Autotuning כדי לגשר על הפער הזה, ומתאים את גודל החוצצים כך שלא ייווצר צוואר בקבוק מיותר באף אחד מהצדדים.

4. הסינתזה: מה קורה בתוך ה-Kernel?

כל הפעולות האלו מתנקזות למבנה הנתונים tcp_sock. כשהשולח מכין חבילה :

1. הוא בודק את החלון האפקטיבי (המינימום בין השניים).
2. הוא מושה נתונים מה-sk_write_queue (חלון השליחה).
3. הוא מחשב את ה-Sequence Number הבא.
4. הוא מפעיל את טיימר ה-RTO (או ה-TLP) כדי לוודא שמה שיצא – אכן הגיע.

למה זה "ריקוד"?

כי שלושת הגורמים האלו משפיעים זה על זה במעגל סגור. אם ה-cwnd גדל, אנחנו צריכים יותר זיכרון ב-sending buffer.
אם הנמען איטי וה-rwnd קטן, ה-cwnd יקפא (כי אין טעם להאיץ אם היעד לא קולט).
הטיימרים (כמו ה-RTO) הם אלו שמוודאים שהריקוד לא נעצר בגלל "קריסה" של שחקן אחד במגרש.

המסקנה:

הבנת ה-TCP ב-Linux 6.14 היא הבנה של ניהול משאבים דינמי.
ה-Sending Window הוא הדירוג הסופי שקובע כמה נתונים ישלטו ברשת, והוא תוצר של פשרה בין אילוצי הזיכרון (בקרת זרימה) לבין אילוצי המרחב-זמן של הרשת (בקרת עומסים).